Design Document

eSim Automated Tool Manager

Screening Task	Task 5 — Tool Manager (CSE and related fields)
Project	eSim Automated Tool Manager
Implementation	Python / Command-Line Interface
Primary Demonstration	Ngspice 47 on Windows
Submitted By	Sumit Kumar Jha
Program	B. Tech — Computer Science & Engineering
Institute	Dronacharya Group of Institutions
University	Dr. A.P.J. Abdul Kalam Technical University (AKTU)
Submission	FOSSEE Semester Long Internship — Autumn 2026
Submission Date	23 August 2026

This document describes the architecture, implementation approach, testing, limitations, and future scope of the screening-task prototype.

Document Overview: -

1. Introduction
2. Problem Statement
3. Objectives
4. Scope
5. Requirements Addressed
6. System Architecture
7. Module Breakdown
8. Tool Installation Workflow
9. Version Checking
10. Dependency Checking
11. Configuration Design
12. Error Handling and Logging
13. Cross-Platform Design
14. Testing and Validation
15. User Interface
16. Security and Reliability Considerations
17. Limitations
18. Future Enhancements
19. Conclusion

1. Introduction: -

The eSim Automated Tool Manager is a Python-based command-line prototype designed to simplify the management of external tools used by eSim. eSim integrates external applications and libraries, and manually checking their installation status, versions, and dependencies can be inconvenient for users.
The proposed tool manager provides a modular mechanism to detect tools, install supported tools, verify installations, check versions, inspect required dependencies, and record operations through logs. The prototype focuses on Ngspice and KiCad and demonstrates automated Ngspice installation on Windows using an official distribution fallback when the package is not available through WinGet.

2. Problem Statement: -

External tools used with eSim may need to be installed and maintained separately. Manual installation can lead to missing dependencies, incorrect versions, configuration issues, and inconsistent environments.
The objective is to create an Automated Tool Manager that reduces this manual effort by providing a single interface for tool discovery, installation, verification, dependency checking, and version information.

3. Objectives: -

- Detect whether supported external tools are installed.
- Provide a simple command-line interface for tool management.
- Automate installation where a supported package source is available.
- Provide an official-source fallback for Ngspice on Windows.
- Automatically download and extract the official Ngspice archive.
- Discover the installed executable and verify the installation.
- Check tool versions.
- Check essential development dependencies such as Python and Git.
- Maintain logs of important operations and errors.
- Keep the implementation modular so additional tools and package managers can be added later.

4. Scope: -

The current proof-of-concept targets Windows and Linux-oriented architecture, with Windows being the primary tested environment. Ngspice is the main end-to-end demonstrated tool, while KiCad is included in the tool registry and detection workflow.

The prototype intentionally focuses on the screening-task requirements that can be demonstrated reliably within the available development time. Automatic update/upgrade management, a graphical interface, macOS support, and complete environment-variable registration are considered future enhancements.

5. Requirements Addressed: -

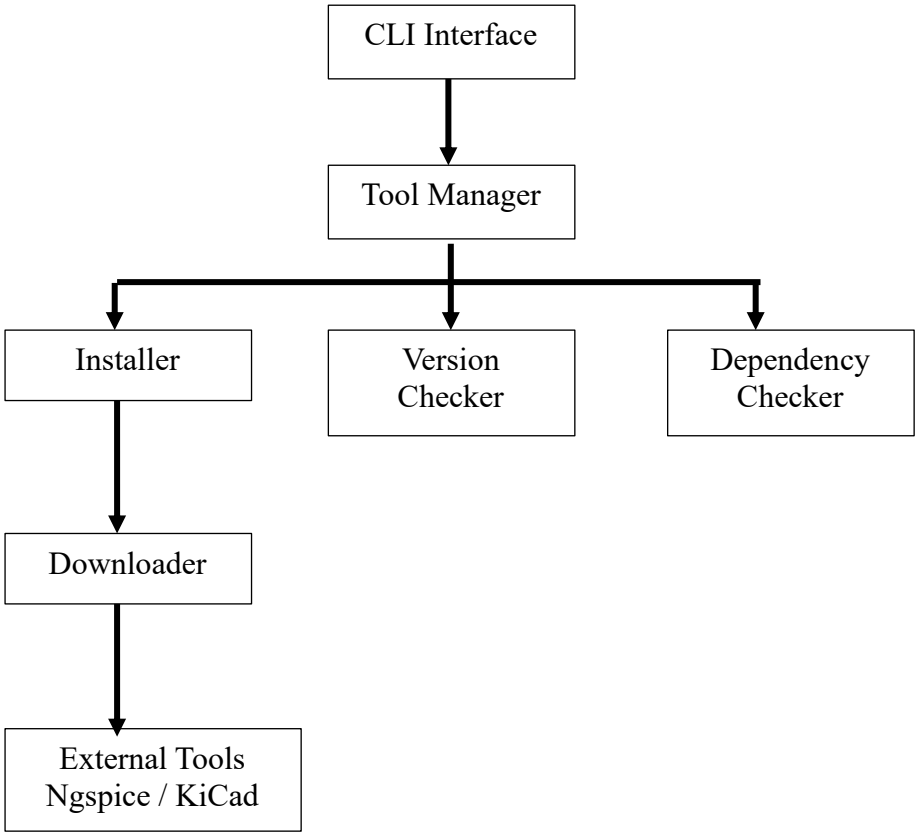
The implementation focuses on three screening requirements:

- Requirement 1 — Tool Installation Management: Implemented through package-manager detection, WinGet checking, official Ngspice fallback, automatic download, archive extraction, executable discovery, and installation verification.
- Requirement 4 — Dependency Checker: Implemented with checks for Python and Git and clear status feedback.
- Requirement 5 — User Interface: Implemented as a command-line interface that allows users to list tools, check versions, install tools, and check dependencies. Logging and error feedback are also provided.

6. System Architecture: -

The system follows a modular architecture. The CLI receives user actions and delegates them to the Tool Manager. The Tool Manager coordinates installation, version checking, dependency checking, downloading, and logging.

Main flow:



7. Module Breakdown: -

- **main.py** → Provides the interactive command-line menu and handles user input.
- **tool_manager.py** → Acts as the central coordinator. It loads tool configuration, lists tools, invokes installation, performs verification, and requests dependency checks.
- **installer.py** → Detects the operating system and manages package-based installation. On Windows it checks WinGet and, when Ngspice is unavailable in the WinGet source, invokes the official-source fallback.
- **downloader.py** → Downloads the official Ngspice Windows archive and extracts the .7z package using py7zr.
- **version_checker.py** → Checks whether commands exist in PATH, recursively searches managed installation directories for executables, and executes version commands.
- **dependency_checker.py** → Checks required local dependencies such as Python and Git and reports their availability.
- **logger.py** → Records initialization, installation, search, verification, dependency, and error events in the project logging system.
- **config/tools.json** →Stores tool metadata, package-manager information, package identifiers, executable names, installation roots, official download sources, and version commands.
- **tests/test_manager.py** → Contains automated tests for core manager functionality.

8. Tool Installation Workflow: -

The Ngspice installation workflow on Windows is:

1. The user selects Install Tool from the CLI.
2. The manager identifies Ngspice from the configuration.
3. The installer checks whether WinGet is available.
4. The installer searches for Ngspice in the configured WinGet source.
5. If Ngspice is unavailable through WinGet, the manager uses the official distribution fallback.
6. The official Ngspice Windows 64-bit archive is downloaded.
7. The .7z archive is extracted into the managed eSimTools directory.
8. The manager recursively searches for ngspice_con.exe.
9. The executable is used for command-line version verification.
10. The installation status and version are reported and logged.

9. Version Checking: -

Version checking supports two approaches.

First, commands available in the system PATH can be checked using shutil.which().

Second, tools installed in the manager-controlled directory can be discovered recursively. This is important for Ngspice because the extracted package contains the executable in a nested directory.

For the demonstrated Ngspice installation, the discovered console executable is:
C:\\Users\\\\eSimTools\\Ngspice\\Spice64\\bin\\ngspice_con.exe

The verified installed release during testing was Ngspice 47.

10. Dependency Checking: -

The dependency checker provides a simple environment health check. The demonstrated checks include:

- Python — required to run the manager.
- Git — useful for repository and development workflows.

The CLI reports each dependency as Installed or Not Installed. The checker can be extended later to include platform-specific libraries and tool-specific dependencies.

11. Configuration Design: -

Tool definitions are separated from the Python implementation and stored in config/tools.json. This makes the system easier to extend without rewriting the core logic.

A tool entry can contain:

- Display name and description
- Windows package manager and package ID
- Linux package manager and package name
- Official fallback URL
- Executable name
- Managed installation root
- Version command

This configuration-driven design supports future addition of tools such as additional eSim dependencies.

12. Error Handling and Logging: -

The application handles common failure conditions such as unavailable package managers, unavailable packages, download errors, extraction errors, timeouts, missing executables, and version-check failures.

Meaningful messages are returned to the CLI instead of silently failing. Important operations are written to the log so that installation and debugging activity can be reviewed after execution.

13. Cross-Platform Design: -

The installer detects the operating system using Python's platform module.

Windows:

- WinGet is checked when applicable.
- Ngspice uses an official distribution fallback when the package is unavailable through WinGet.

Linux:

- The configuration supports apt-based package installation for configured tools.

macOS:

- The operating-system detection structure is present, but a complete macOS installer is not implemented in the current prototype.

The design therefore provides a foundation for additional package managers such as Homebrew and other platform-specific mechanisms.

14. Testing and Validation: -

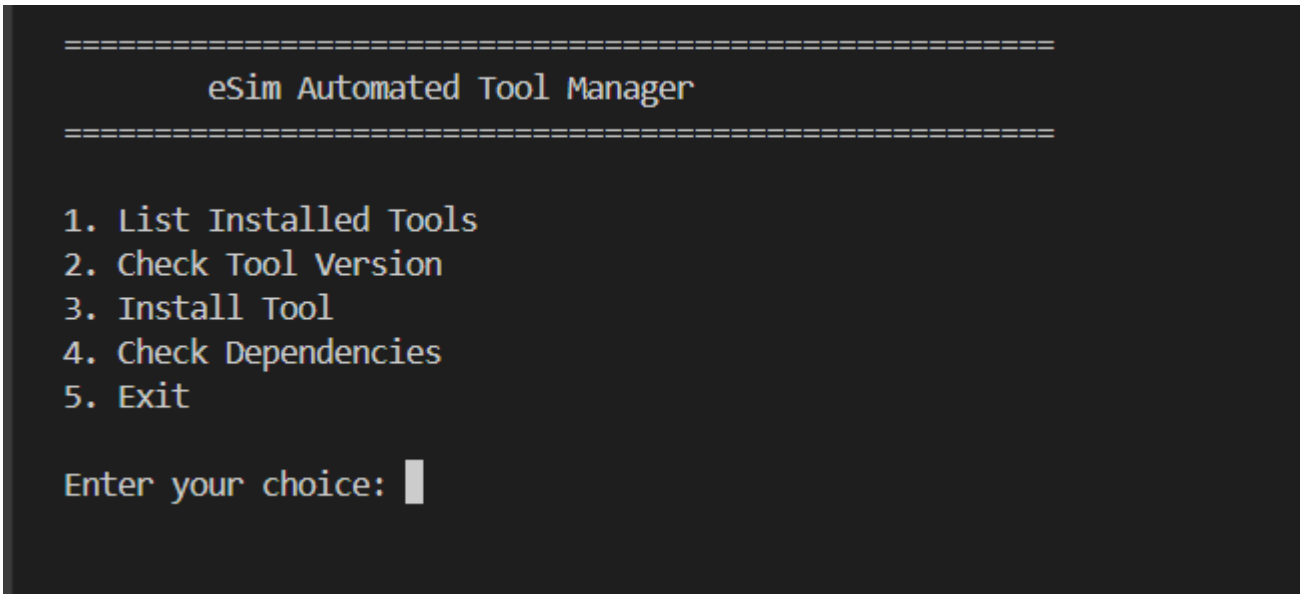
The prototype was tested in a Windows development environment.

Validation performed:

- Python source files were syntax-checked using py_compile.
- Automated project tests completed successfully with the result “All tests passed.”
- WinGet availability was checked.
- Ngspice was confirmed to be unavailable through the configured WinGet source.
- The official Ngspice archive was downloaded automatically.
- The archive was extracted successfully.
- ngspice.exe and ngspice_con.exe were located.
- Ngspice 47 was launched successfully.
- The console executable was used for version verification.
- Python and Git dependency checks reported Installed.

15. User Interface: -

The prototype uses a simple CLI menu:



1. List Installed Tools 2. Check Tool Version 3. Install Tool 4. Check Dependencies 5. Exit

16. Security and Reliability Considerations: -

The installer executes system-level package commands only from configured tool metadata rather than accepting arbitrary commands from the user.

Downloads use a predefined official source for the demonstrated Ngspice fallback. Future versions should add archive checksum/signature verification, stricter URL validation, elevated-permission handling, rollback support, and safer package-manager abstraction.

17. Limitations: -

- Automatic update/upgrade functionality is not implemented in the current prototype.
- Complete GUI support is not implemented.
- macOS installation is not implemented.
- The demonstrated end-to-end automatic installation focuses on Ngspice.
- Full eSim-specific environment configuration is reserved for future work.
- The current dependency checker covers a small set of core dependencies.

18. Future Enhancements: -

- Automatic update and upgrade detection.
- Version constraints and compatibility rules.
- SHA-256/checksum verification for downloaded packages.
- Support for additional package managers such as Chocolatey and Homebrew.
- Automatic PATH/environment configuration.
- Tool rollback and uninstall support.
- Graphical interface using Tkinter or PyQt.
- More eSim external tools and dependency definitions.
- Structured JSON logs and exportable diagnostic reports.
- Automated CI testing across Windows and Linux.

19. Conclusion: -

The eSim Automated Tool Manager demonstrates a practical Python-based approach to managing external eSim tools. The prototype successfully combines a CLI, configuration-driven tool definitions, package-manager detection, official-source fallback, automatic Ngspice download and extraction, executable discovery, version verification, dependency checking, and logging.

The implementation satisfies the screening-task objective of producing a working proof of concept and provides a modular foundation for future update, configuration, and cross-platform capabilities.

Appendix A — Requirement Coverage Matrix

Requirement	Status	Evidence in Prototype
1. Tool Installation Management	**Implemented**	WinGet + Ngspice fallback; download, extract & verify.
2. Update and Upgrade System	**Future Enhancement**	Version comparison & updates planned.
3. Configuration Handling	**Partially Implemented**	tools.json based tool configuration.
4. Dependency Checker	**Implemented**	Python & Git checks.
5. User Interface	**Implemented**	CLI-based tool management.
6. Additional Features	**Partially Implemented / Future**	Windows/Linux; more platforms planned.

Note:

The matrix distinguishes implemented functionality from planned enhancements to accurately represent the current proof-of-concept implementation.